

deepin

wlframe

C语言跨平台GUI库

<https://github.com/zzxyb/wlframe>

图形窗管研发组-肖耀兵

以用为本 拥抱创新

www.deepin.org

Content

目录

01

背景

wlframe因何而生

02

介绍

wlframe定位和策略

03

架构

将“像素如何产生和提交”分开，极致的渲染美学

04

现状

万行框架代码，投入主要集中在基础链路

05

未来构想

真实需求驱动架构分层

06

总结

wlframe下一步要回答“应用为何愿意使用”

01

背景

UI 框架的难点，是让整条链路持续正确。



01 原生 UI 的四类复杂度

■ 以用为本 拥抱创新

协议、资源、界面状态和生命周期
共同决定最终体验



02 Wayland 把更多客户端责任暴露出来

■ 以用为本 拥抱创新

应用 / UI 框架

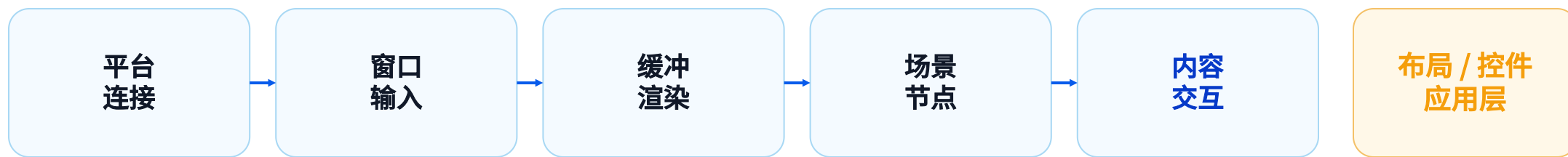
- 01 组织内容、处理输入、决定何时重绘
- 02 创建并维护缓冲区
- 03 提交 surface 状态，遵循 configure、scale 和 frame callback

Wayland 合成器：组合客户端 surface，输出到屏幕

“画出来”只是起点，“按协议稳定运行”才是可用。

03 wlframe 的演进路径

■ 以用为本 拥抱创新



先建立可控、可观察的运行路径，再让真实应用推动高层抽象。

02

介绍

wlframe定位和策略



01 定位：可组合、可控制的原生界面基础

■ 以用为本 拥抱创新

01

语言边界：C API，方便接入系统代码，也为 FFI 保留入口

02

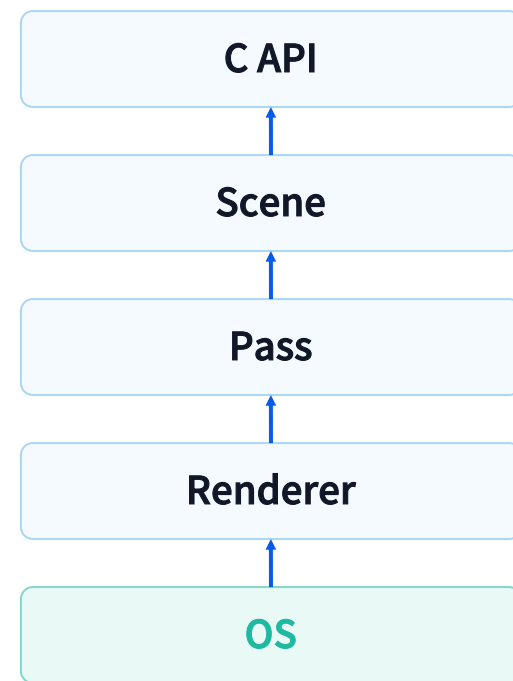
界面模型：持久场景树，应用通过 setter 更新节点状态

03

渲染策略：Pass 表达绘制意图，Renderer 对接 CPU / GPU 后端

04

平台策略：公共语义稳定，平台差异留在具体实现



适合原生工具、桌面组件和需要深度定制的图形界面

02 当前能力边界

■ 以用为本 拥抱创新

已有基础	尚未形成完整能力
窗口、输入、场景树	通用布局系统
文本、图片、SVG、图形节点	稳定控件库与组件模型
GL ES / Pixman 场景链路	完整文本编辑与输入法体验
Damage、可见性、交换链	无障碍与成熟工具链
动画基类与组合	跨平台一致的端到端体验

现在更适合验证架构与目标场景，不宜把“有模块”误读为“已成熟”。

03 最小应用路径

■ 以用为本 拥抱创新



```
backend = wlf_backend_autocreate();  
window  = /* 创建平台窗口 */;  
scene   = wlf_scene_create(window);  
/* 创建节点、注册事件，然后进入事件循环 */
```

03

架构

极致的渲染美学



01 wlframe 整体架构

■ 以用为本 拥抱创新

Window • 窗口 / 输入 / 焦点

Scene / Node • 层级 / 状态 / Damage

Pass • 绘制意图

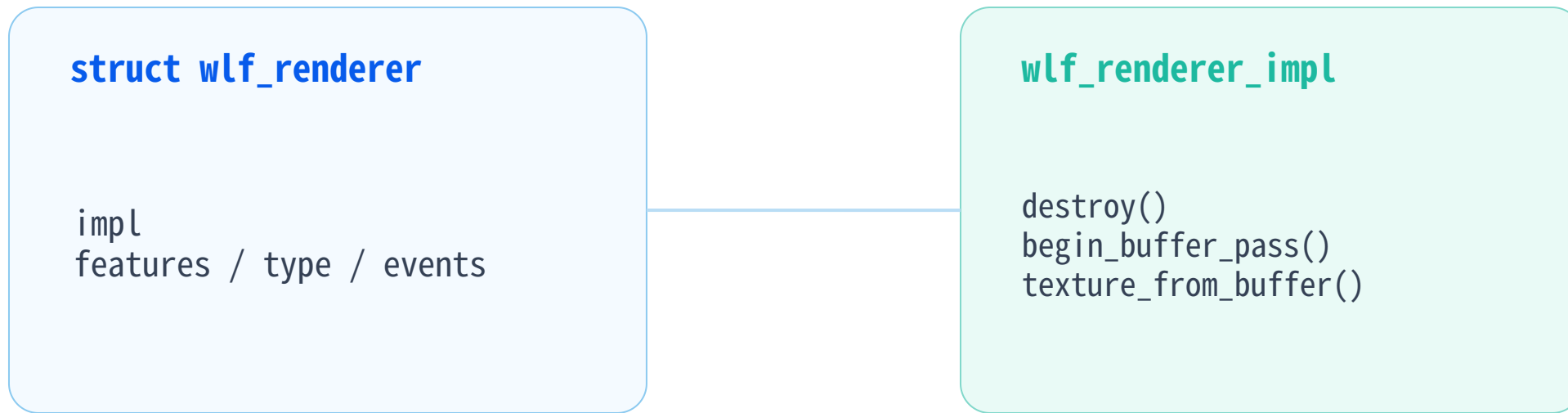
Texture / Buffer / Swapchain

Platform Adapter • Wayland / Win32 / macOS

操作系统 / 合成器

02 C 接口的多态实现

■ 以用为本 拥抱创新

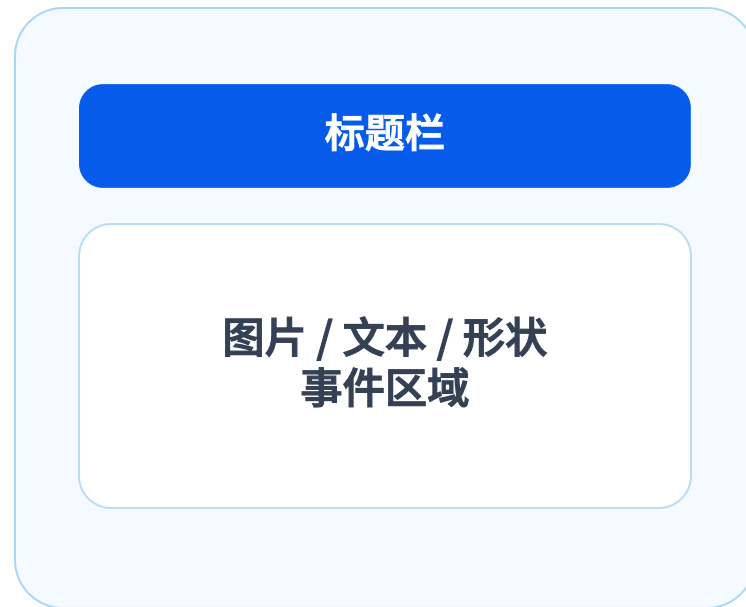


抽象描述“要做什么”，具体结构体承担“如何做到”。

03 场景树与坐标空间

■ 以用为本 拥抱创新

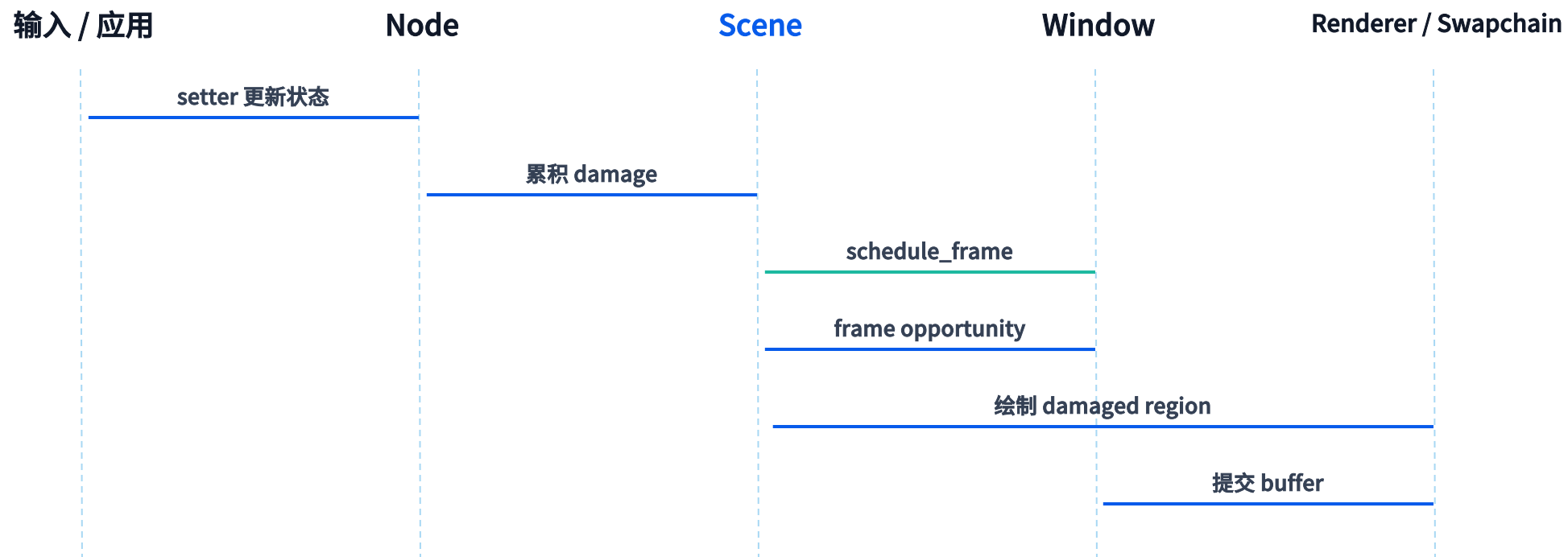
```
Window
├── Scene root
│   ├── 客户端装饰层
│   │   ├── Titlebar / buttons / resize handles
│   │   └── 应用内容树
│   │       ├── Card tree
│   │       │   ├── Rect node
│   │       │   ├── Texture node
│   │       │   ├── Text node
│   │       │   └── Event node
```



节点状态：位置、尺寸、透明度、可见区域、焦点策略与输入穿透区域

04 状态更新与帧提交

■ 以用为本 拥抱创新

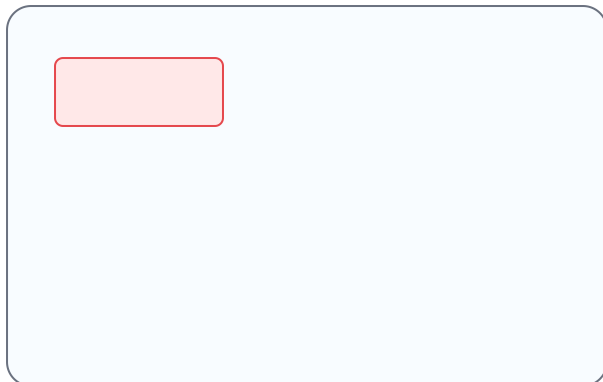


多次 setter 可以合并成一帧；成功提交后再清理已处理的 Damage。

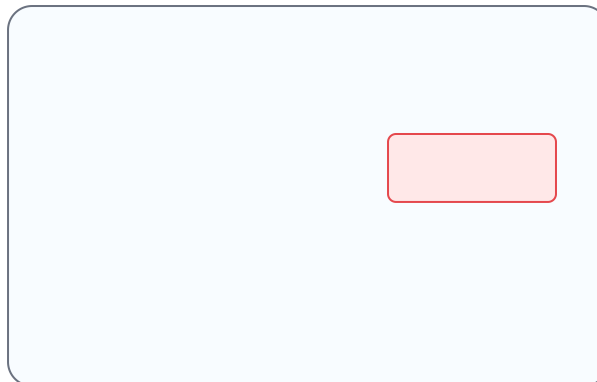
05 Damage 与缓冲区复用

■ 以用为本 拥抱创新

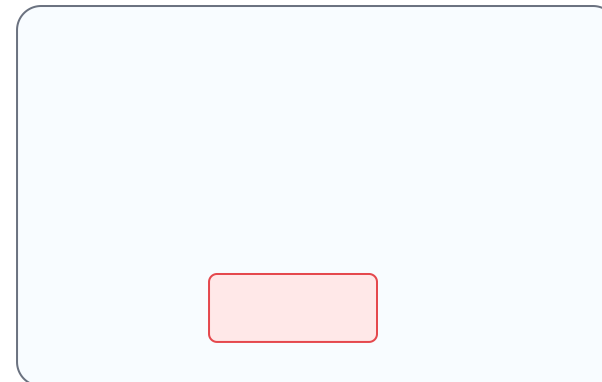
t0 • Buffer A



t1 • Buffer B



t2 • Buffer A 复用



若 t2 只绘底部，Buffer A 仍缺少 t1 的右侧变化。

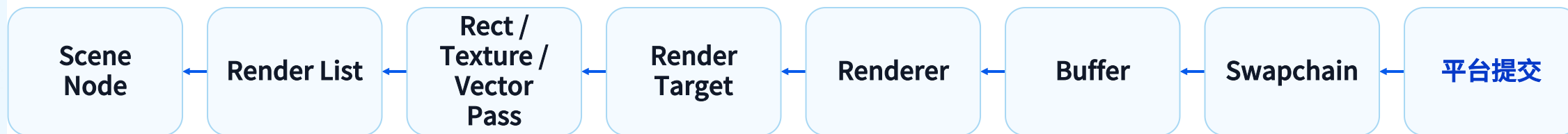
$\text{render_damage} = \text{current_damage} \cup \text{previous_damage}$

```
[07:40:26.252941] {Display Queue} wl_display#1.delete_id(42)
[07:40:26.252981] {Default Queue} wl_callback#42.done(5502922)
[07:40:26.253110] {mesa egl surface queue} wl_callback#34.done(6636)
[07:40:26.253126] {mesa egl surface queue} wl_buffer#41.release()
[07:40:26.253192] {Default Queue} -> wl_surface#25.frame(new id wl_callback#34)
[07:40:26.253364] {mesa egl surface queue} -> wl_surface#25.attach(wl_buffer#41, 0, 0)
[07:40:26.253384] {mesa egl surface queue} -> wl_surface#25.damage_buffer(142, 237, 182, 150)
[07:40:26.253396] {mesa egl surface queue} -> wl_surface#25.commit()
[07:40:26.253403] {mesa egl surface queue} -> wl_display#1.sync(new id wl_callback#42)
[07:40:26.253964] {Display Queue} wl_display#1.delete_id(42)
[07:40:26.269311] {Display Queue} wl_display#1.delete_id(34)
[07:40:26.269336] {Default Queue} wl_callback#34.done(5502939)
[07:40:26.269434] {mesa egl surface queue} wl_callback#42.done(6636)
[07:40:26.269442] {mesa egl surface queue} wl_buffer#39.release()
[07:40:26.269490] {Default Queue} -> wl_surface#25.frame(new id wl_callback#42)
[07:40:26.269607] {mesa egl surface queue} -> wl_surface#25.attach(wl_buffer#39, 0, 0)
[07:40:26.269620] {mesa egl surface queue} -> wl_surface#25.damage_buffer(144, 237, 182, 150)
[07:40:26.269626] {mesa egl surface queue} -> wl_surface#25.commit()
[07:40:26.269631] {mesa egl surface queue} -> wl_display#1.sync(new id wl_callback#34)
[07:40:26.270023] {Display Queue} wl_display#1.delete_id(34)
[07:40:26.286337] {Display Queue} wl_display#1.delete_id(42)
[07:40:26.286377] {Default Queue} wl_callback#42.done(5502956)
[07:40:26.286509] {mesa egl surface queue} wl_callback#34.done(6636)
[07:40:26.286525] {mesa egl surface queue} wl_buffer#41.release()
[07:40:26.286593] {Default Queue} -> wl_surface#25.frame(new id wl_callback#34)
[07:40:26.286788] {mesa egl surface queue} -> wl_surface#25.attach(wl_buffer#41, 0, 0)
[07:40:26.286810] {mesa egl surface queue} -> wl_surface#25.damage_buffer(145, 237, 182, 150)
[07:40:26.287026] {mesa egl surface queue} -> wl_surface#25.commit()
[07:40:26.287043] {mesa egl surface queue} -> wl_display#1.sync(new id wl_callback#42)
[07:40:26.287451] {Display Queue} wl_display#1.delete_id(42)
[07:40:26.303596] {Display Queue} wl_display#1.delete_id(34)
[07:40:26.303626] {Default Queue} wl_callback#34.done(5502973)
[07:40:26.303738] {mesa egl surface queue} wl_callback#42.done(6636)
[07:40:26.303748] {mesa egl surface queue} wl_buffer#39.release()
[07:40:26.303807] {Default Queue} -> wl_surface#25.frame(new id wl_callback#42)
[07:40:26.303939] {mesa egl surface queue} -> wl_surface#25.attach(wl_buffer#39, 0, 0)
[07:40:26.303951] {mesa egl surface queue} -> wl_surface#25.damage_buffer(147, 237, 182, 150)
[07:40:26.303959] {mesa egl surface queue} -> wl_surface#25.commit()
[07:40:26.303964] {mesa egl surface queue} -> wl_display#1.sync(new id wl_callback#34)
[07:40:26.304316] {Display Queue} wl_display#1.delete_id(34)
[07:40:26.319056] {Display Queue} wl_display#1.delete_id(42)
[07:40:26.319077] {Default Queue} wl_callback#42.done(5502989)
[07:40:26.319125] {mesa egl surface queue} wl_callback#34.done(6636)
[07:40:26.319130] {mesa egl surface queue} wl_buffer#41.release()
[07:40:26.319162] {Default Queue} -> wl_surface#25.frame(new id wl_callback#34)
[07:40:26.319247] {mesa egl surface queue} -> wl_surface#25.attach(wl_buffer#41, 0, 0)
[07:40:26.319256] {mesa egl surface queue} -> wl_surface#25.damage_buffer(148, 237, 182, 150)
[07:40:26.319265] {mesa egl surface queue} -> wl_surface#25.commit()
[07:40:26.319270] {mesa egl surface queue} -> wl_display#1.sync(new id wl_callback#42)
[07:40:26.319450] {Display Queue} wl_display#1.delete_id(42)
```



06 场景渲染与资源提交

■ 以用为本 拥抱创新



绘制意图

- 01 Pass 统一节点的绘制入口
- 02 后端实现决定实际 API 调用

资源提交

- 01 Texture / Buffer 覆盖绘制与提交生命周期
- 02 Swapchain 轮换可写缓冲区

07 输入状态与生命周期

■ 以用为本 拥抱创新



命中、抓取、焦点、取消和目标销毁，必须遵守同一套生命周期。

08 内容模型

■ 以用为本 拥抱创新

UTF-8 文本

Pango / Cairo

多种图片格式

图像解码

SVG

解析与图元转换

Rect / Circle / Path

几何与 coverage

Texture / Text Node

SVG Node

Vector Nodes

Scene

```
graph TD; A[UTF-8 文本  
Pango / Cairo] --> D[Texture / Text Node]; B[多种图片格式  
图像解码] --> D; C[SVG  
解析与图元转换] --> E[SVG Node]; F[Rect / Circle / Path  
几何与 coverage] --> G[Vector Nodes]; D --> H[Scene]; E --> H; G --> H;
```

04

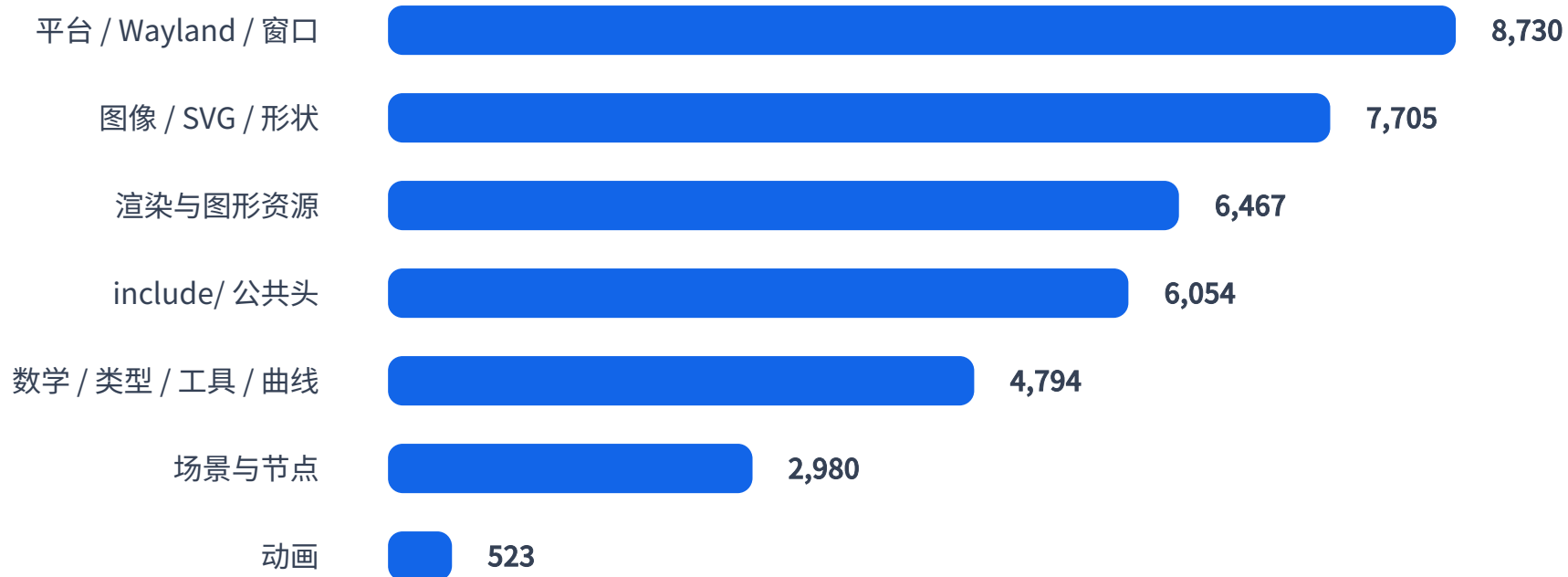
现状

万行框架代码筑基



01 代码投入分布

■ 以用为本 拥抱创新



框架源码: 353 个文件 / 37,253 行 • 示例: 5,976 行

03 主线能力矩阵

■ 以用为本 拥抱创新

垂直链路	窗口 / 平台	场景 Pass	资源 / 提交	当前定位
Wayland + GLES	●	●	●	主要演示路径
Wayland + Pixman	●	●	●	主要演示路径
Wayland + Vulkan	●	◐	●	建设中
macOS + Metal	◐	○	◐	建设中
Windows + D3D12（主线）	◐	○	◐	建设中

● 已接入 ◐ 部分基础 ○ 关键链路待补齐

分支上的实验能力与主线状态分开表达。

04 项目演进

■ 以用为本 拥抱创新

2024

2025

2026 上半年

2026-09

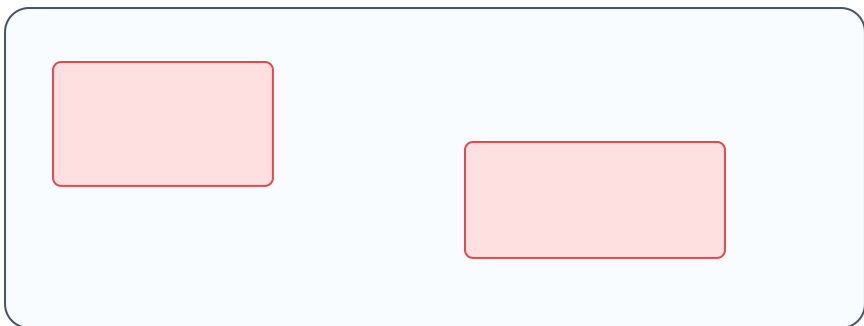


下一步的重点：让已有类型在统一调度和生命周期中协作。

05 Damage 调试演示

■ 以用为本 拥抱创新

```
WLF_RENDERER=pixman \  
WLF_SCENE_DEBUG_DAMAGE=highlight \  
./build/examples/window/scene_damage_test
```



Damage highlight

01

移动图形：旧位置与新位置同时标红

02

快速连续更新：多个变化合并到下一帧

03

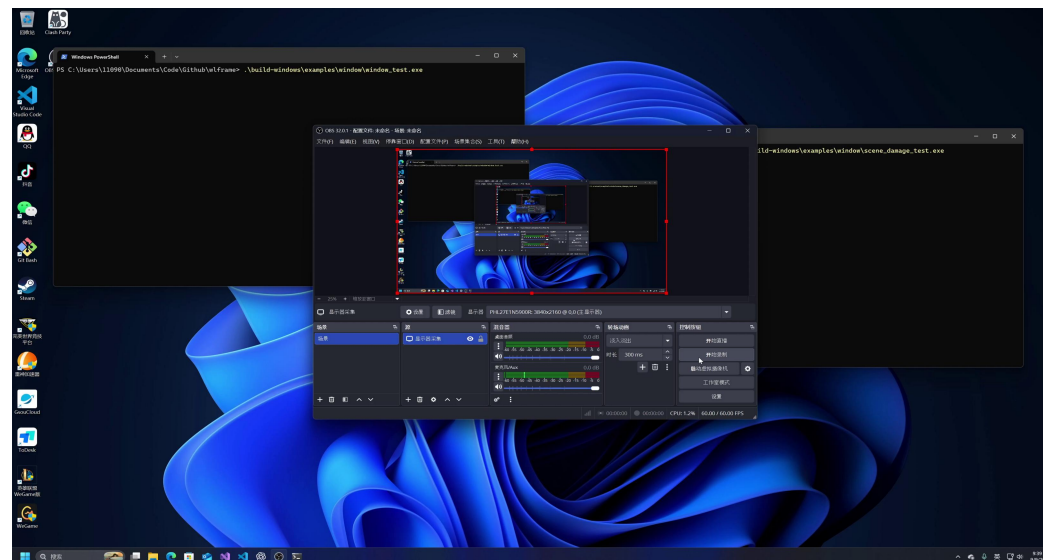
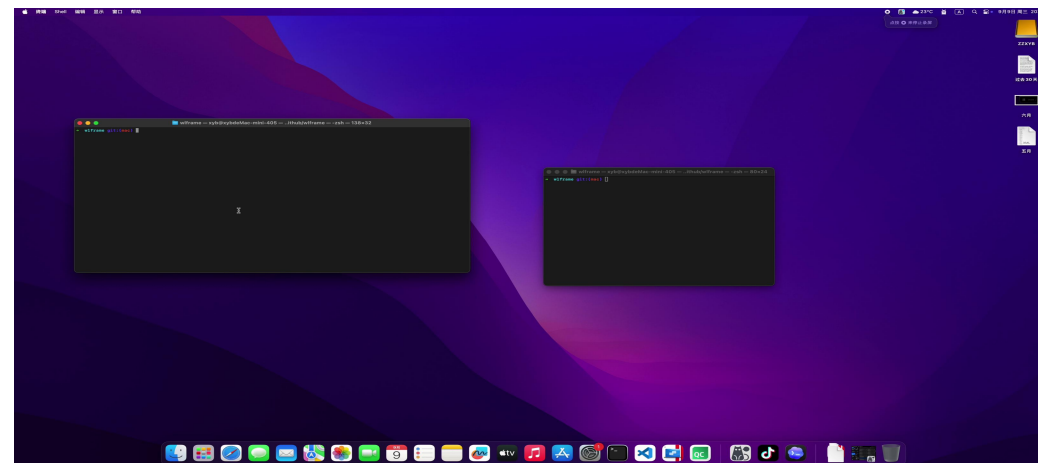
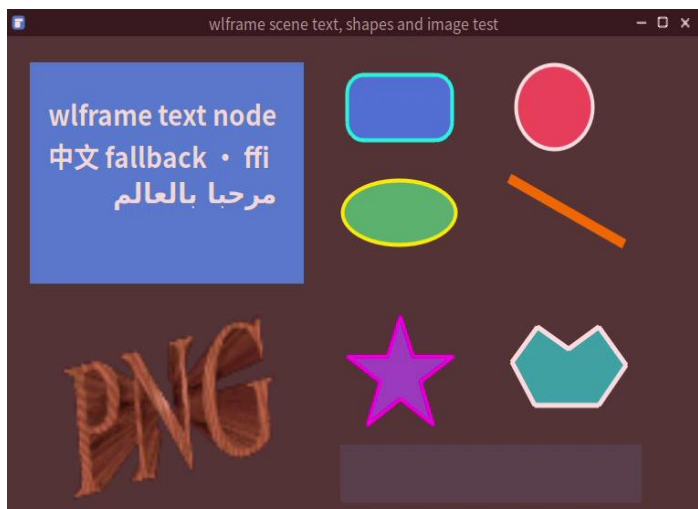
切换 Pixman / GLES：同一场景验证后端一致性

04

调整窗口：观察整窗重绘与缩放处理

06 效果预览

■ 以用为本 拥抱创新



05

未来构想

真实需求驱动架构分层



01 公共语义与平台适配

■ 以用为本 拥抱创新

公共语义: Window · Input · Scene · Pass · Texture

Wayland Adapter

Wayland + EGL / SHM

Win32 Adapter

Win32 + DXGI / D3D12

macOS Adapter

AppKit + Metal

02 Windows 分支的垂直链路验证

■ 以用为本 拥抱创新

窗口

Win32 toplevel · 输入 · per-monitor DPI · 输出热更新

软件路径

Windows Pixman pipeline · scene smoke test

文本

GDI 栅格化到 DirectWrite · committed / preedit · IME

帧节奏

invalidation · frame pacing · DWM 同步

GPU 路径

DXGI swapchain · D3D12 solid / texture pass · dirty rect

验证的是“窗口、输入、文本、场景、呈现”整条路径。

03 macOS 实现路径

■ 以用为本 拥抱创新

已有基础	下一条可验收链路
macOS backend 与主题基础	AppKit 原生窗口创建与事件循环
Metal renderer / device 基础	可呈现的 swapchain / drawable 管理
公共 Scene / Node / Pass 模型	Metal 场景 Pass 与纹理上传
跨平台 Window / Input 契约	键鼠、触摸板、缩放和文本输入
平台文本抽象	CoreText / Core Graphics 文本路径

创建窗口 绘制矩形 显示文字 / 图片 处理输入 缩放 / 动画 可靠退出

04 跨平台验收标准

■ 以用为本 拥抱创新

01 构建与启动

02 创建 / 销毁窗口

03 矩形 / 图片 / 文本

04 键鼠 / 触摸输入

05 文本与输入法

06 缩放与多显示器

07 Damage 与动画帧

08 休眠、恢复与退出

“能编译”是起点；“相同语义可重复通过”才是跨平台。

06

总结

应用为何愿意使用



01 核心结论

■ 以用为本 拥抱创新

01 它是什么

用 C 组织窗口、场景、渲染、资源与输入的跨平台 UI 框架探索

02 它做到哪

主线以 Wayland + GLES / Pixman 为核心，Windows/mac 分支已验证更完整的垂直路径

03 它往哪走

用真实应用推动布局、控件、文本、无障碍、性能与平台一致性

框架价值在于：状态变化能否被稳定、可测地送到每个平台的屏幕。

Thanks For Watching

深度科技社区



微信公众号

